

SC102 L03：设计模式——详细学习笔记

涵盖：单例模式 | 指令模式 | 工厂模式 | 观察者模式 | 策略模式 | 对象池 | ECS 模式

1 什么是设计模式？

[p.4]

设计模式（Design Patterns）是编写代码时可以遵循的**模板或思想**，它们是前人在遇到常见问题时总结出的最优解决方案。

经典问题示例：

- **撤回功能（Ctrl+Z）：**同样的输入可以撤回各种不同的操作，这是怎么实现的？如果把每个操作抽象成一个”指令”，记录执行过的指令，撤回时反向执行即可——这就是指令模式。
- **特效性能优化：**每帧生成跑步特效，不断 Create/Destroy 会严重影响性能。将创建过的特效存入”池”中复用，用 SetActive 代替 Destroy——这就是对象池模式。

设计模式的核心价值不在于发明新东西，而在于给已知问题提供**经过验证的标准解法**，让代码更易维护、更可扩展。

2 单例模式（Singleton Pattern）

2.1 核心概念

[p.5]

单例模式是最简单、最常用、也最容易被滥用的设计模式。它确保两点：

1. 始终只允许有一个该类的实例（单一性）
2. 对于该实例有一个全局的访问点（全局性）

游戏中许多”Manager”类（GameManager、AudioManager、UIManager 等）都天然需要这两个特征。

2.2 基础实现

```
public class Singleton {
    private static Singleton instance;           // 私有的静态实例引用
    private Singleton() { }                     // 私有构造函数，阻止外部new
    public static Singleton Instance {
        get {
            if (instance == null)
                instance = new Singleton();    // 延迟实例化（懒加载）
            return instance;
        }
    }
}
```

为什么满足单例的两点特性？

- **单一性**：构造函数为 `private`，外部无法用 `new` 创建第二实例；`Instance` 属性中只有 `instance == null` 时才创建
- **全局性**：`public static` 的 `Instance` 属性可以从任何地方通过 `Singleton.Instance` 访问

2.3 泛型单例基类

[p.6-7]

为每个单例类都写一段相同的单例代码不够优雅。我们可以创建一个泛型单例基类：

前置知识：

- `protected` 访问修饰符：只能被该类及其子类访问（介于 `private` 和 `public` 之间）
- `??` 运算符（null-coalescing）：`a ?? b` 等价于 `a != null ? a : b`（`a` 不为 `null` 返回 `a`，否则返回 `b`）
- Lambda 表达式属性：`public int MyInt => myInt < 0 ? 0 : myInt`；是 `getter` 的简写

通用泛型单例实现：

```
public class Singleton<T> where T : class, new() {
    protected Singleton() { }                // protected允许子类访问
    private static T _inst = null;
    public static T instance => _inst ?? (_inst = new T());

    public static void Clear() {
        _inst = null;
    }
}
```

```
}
```

```
// 使用：只需要继承即可
```

```
public class GameManager : Singleton<GameManager> {  
    public int score = 0;  
    public void AddScore(int points) { score += points; }  
}
```

```
// 在任何地方
```

```
GameManager.instance.AddScore(10);
```

代码逐行解析：

- `where T : class, new()` —— `T` 必须是引用类型且有公共无参构造函数（泛型约束）
- `protected Singleton() {}` —— 构造函数 `protected`，子类可以调用，外部不能直接 `new`
- `public static T instance => _inst ?? (_inst = new T());` —— `Lambda` 属性，首次访问时如果 `_inst` 为 `null` 则创建新实例
- `public static void Clear()` —— 清除实例（场景切换时可能需要）

2.4 单例模式的优缺点

[p.8]

优点：

- **单一性**：始终只有一个实例，方便管理，不会出现多个 `Manager` 造成混乱
- **便捷性**：全局访问点，任何地方都能访问
- **懒加载**：只在首次需要时实例化，不浪费资源

缺点：

- **全局性增加耦合**：全局访问点使得类之间依赖关系隐蔽，降低可读性，提高出错率。很多情况下 `static` 字段和方法是更好的选择
- **单点故障**：单例出错时整个系统可能崩溃

使用建议：使用前想想是否有更好的实现方式（依赖注入、`static` 工具类、`ScriptableObject` 等）。小项目中尽情使用也没问题。

3 指令模式 (Command Pattern)

3.1 核心概念

[p.9]

指令模式是”Ctrl+Z”（撤回功能）的灵魂。它将每个操作抽象成一个指令对象，每个指令有”执行”和”撤销”两个操作。记录执行过的指令，一步步反向撤销即可。

3.2 实现框架

```
public interface ICommand {
    void Execute(); // 执行指令
    void Undo();    // 撤销指令
}

public class MoveCommand : ICommand {
    private Transform transform;
    private Vector3 direction;
    private float distance;

    public MoveCommand(Transform tf, Vector3 dir, float dist) {
        transform = tf;
        direction = dir;
        distance = dist;
    }

    public void Execute() {
        transform.position += direction * distance;
    }

    public void Undo() {
        transform.position -= direction * distance; // 反向移动
    }
}
```

3.3 指令模式的五个组成部分

[p.10]

1. 指令接口 (Command Interface/ICommand): 定义 Execute 和 Undo 方法
2. 具体指令类 (Concrete Command): 实现具体操作和撤销逻辑

3. 调用者 (Invoker): 类似一个 Manager, 存储和执行指令
4. 接收者 (Receiver): 真正执行指令的对象 (如 Transform、GameObject)
5. 使用者 (Client): 创建指令并交给 Invoker 执行

完整的指令管理器实现:

```
public class CommandInvoker {
    private Stack<ICommand> commandHistory = new Stack<ICommand>();

    public void ExecuteCommand(ICommand command) {
        command.Execute();
        commandHistory.Push(command); // 记录到历史中
    }

    public void Undo() {
        if (commandHistory.Count > 0) {
            ICommand lastCommand = commandHistory.Pop();
            lastCommand.Undo();        // 撤销最近一个指令
        }
    }
}
```

3.4 优缺点

[p.11]

优点: 将调用者和执行者解耦; 完美支持撤回/重做功能 **缺点:** 每个指令都是一个单独的类, 项目大后可能类数量爆炸

适用场景: 关卡编辑器 (每个编辑操作都是 Command)、回合制游戏的行动序列、宏录制与回放

4 工厂模式 (Factory Pattern)

4.1 简单工厂模式

[p.12]

工厂模式将类的实例化过程包装起来, 让使用者方便且可控地获得实例。

```
public abstract class Animal {
    public abstract string Speak();
}
```

```
public class Dog : Animal {
    public override string Speak() { return "Woof"; }
}

public class Cat : Animal {
    public override string Speak() { return "Meow"; }
}

public static class AnimalFactory {
    public static Animal CreateAnimal(string type) {
        switch (type) {
            case "Dog": return new Dog();
            case "Cat": return new Cat();
            default: throw new NotSupportedException($"Unknown animal: {type}");
        }
    }
}

// 使用
Animal myPet = AnimalFactory.CreateAnimal("Dog");
```

工厂模式的价值：

- 将创建逻辑集中管理，修改时只改一处
- 隐藏具体类型（调用者只知道 Animal，不知道 Dog/Cat）
- 创建过程中可以添加额外逻辑（日志、缓存、验证等）

4.2 抽象工厂模式 (Abstract Factory)

[p.13]

抽象工厂是“生产工厂的工厂”。引入工厂接口，允许多个工厂实现不同系列的创建：

```
public interface IAnimalFactory {
    Dog CreateDog();
    Cat CreateCat();
}

public class RedAnimalFactory : IAnimalFactory {
    public Dog CreateDog() { return new RedDog(); }
    public Cat CreateCat() { return new RedCat(); }
}
```

```
}
```

```
public class BlueAnimalFactory : IAnimalFactory {
    public Dog CreateDog() { return new BlueDog(); }
    public Cat CreateCat() { return new BlueCat(); }
}
```

4.3 两种工厂模式的区别

[p.14]

- **简单工厂**：生产单一产品体系（Cat 或 Dog），把创建过程包装起来。简单直接。
- **抽象工厂**：生产产品族（Red 系列 Dog+Cat vs Blue 系列 Dog+Cat），允许在运行时切换整个产品族。更复杂，适合大系统。

5 观察者模式（Observer Pattern）

[p.15]

观察者模式中有”观察者（Observer）”和”通知者/主题（Subject）”两方。当通知者发生变化时，所有观察者都会收到通知并做出反应。

我们早已用过：C# 的委托与事件系统就是观察者模式的一种实现。Unity 的 UnityEvent（如 Button.onClick）也是观察者模式的体现。

手动实现观察者模式：

```
public class Subject {
    private List<IObserver> observers = new List<IObserver>();

    public void AddObserver(IObserver observer) {
        observers.Add(observer);
    }

    public void RemoveObserver(IObserver observer) {
        observers.Remove(observer);
    }

    public void Notify() {
        foreach (var observer in observers) {
            observer.OnNotify();
        }
    }
}
```

```

}

public interface IObservable {
    void OnNotify();
}

```

这本质上和 `event Action` 做的事情一样——事件就是 C# 内置的观察者模式实现。

6 策略模式 (Strategy Pattern)

[p.16]

策略模式允许在运行时根据情况选择不同的算法来完成任务。

```

// 无策略模式 (if-else 版本)
void TakeDamage() {
    if (InWater())
        TakeDamageInWater();
    else
        TakeDamageOnLand();
}

// 策略模式版本
public interface IDamageStrategy {
    void TakeDamage(float damage);
}

public class WaterDamageStrategy : IDamageStrategy { ... }
public class LandDamageStrategy : IDamageStrategy { ... }

public class Player {
    private IDamageStrategy damageStrategy;

    public void SetStrategy(IDamageStrategy strategy) {
        damageStrategy = strategy;
    }

    public void TakeDamage(float damage) {
        damageStrategy.TakeDamage(damage);
    }
}

```


策略模式的优势：新增策略时不修改已有代码（开闭原则）、策略可运行时动态切换、每个策略独立测试。

7 对象池（Object Pool）

7.1 为什么需要对象池？

[p.18]

在 Unity 中，`Instantiate()` 和 `Destroy()` 是非常消耗性能的操作。对于简单、可复用的物体（如子弹、特效、敌人），可以用 **enable/disable** 代替创建/销毁——用 `SetActive(false)` 隐藏物体而非销毁，用 `SetActive(true)` 重新激活而非创建新物体。

对象池的工作原理：

1. 初始化时预创建一批物体放入”池”中，全部 `SetActive(false)`
2. 需要使用时，从池中找一个空闲（inactive）的物体，激活后返回
3. 用完时，`SetActive(false)` 标记为空闲，放回池中（不销毁）
4. 池中没有空闲物体时才创建新物体

这和 **String Interning**（字符串驻留）的思想非常相似。

7.2 基本对象池实现

[p.19]

```
public class ObjectPool : MonoBehaviour {
    public GameObject objectToPool;
    public int initialPoolSize = 10;
    private List<GameObject> pool;

    void Start() {
        pool = new List<GameObject>();
        for (int i = 0; i < initialPoolSize; i++) {
            GameObject obj = Instantiate(objectToPool);
            obj.SetActive(false);
            pool.Add(obj);
        }
    }

    public GameObject Get() {
        // 先找空闲的
```

```

        for (int i = 0; i < pool.Count; i++) {
            if (!pool[i].activeInHierarchy) {
                pool[i].SetActive(true);
                return pool[i];
            }
        }
        // 池中无空闲则创建新的
        GameObject obj = Instantiate(objectToPool);
        pool.Add(obj);
        return obj;
    }

    public void ReturnToPool(GameObject obj) {
        obj.SetActive(false);
    }
}

```

7.3 更高级的对象池

[p.20]

实际项目中，对象池通常需要处理：

- 多种类型的对象（用 Dictionary<string, Queue<GameObject>> 管理）
- 自动扩容和缩容（根据负载动态调整池大小）
- 回收时重置对象状态（位置、血量等）
- 泛型化，支持任意 Component 类型

8 ECS 模式 (Entity-Component-System)

8.1 ECS 的三个组成部分

[p.21]

- **Entity (实体)**：一个有 ID 的物体，包含一系列 Component。和 GameObject 类似但不自带任何逻辑。
- **Component (组件)**：纯数据结构（没有方法！），只存储数据。如 Position 组件只存储 x 和 y 坐标。
- **System (系统)**：真正的逻辑执行者。一个 System 在所有拥有特定 Component 组合的 Entity 上执行操作。如 MovementSystem 遍历所有同时有 Position 和 Velocity 组件的 Entity，更新位置。

8.2 与 Unity GameObject 体系的关键区别

Unity 并不完全是 ECS 模式！在 Unity 中：

- 数据和逻辑都在 **Component** 里（如 MonoBehaviour 既存数据又写 Update() 逻辑）
- 没有独立的 **System**，每个 Component 自己 Update

ECS 将数据和逻辑彻底分离：Component 只存数据，System 只写逻辑。这种思想叫做面向数据编程（Data-Oriented Programming, DOP），与传统的面向对象编程（OOP）形成对比。

8.3 简易 ECS 实现示例

[p.22]

// Components — 纯数据类

```
class Position {  
    public float x;  
    public float y;  
}
```

```
class Velocity {  
    public float dx;  
    public float dy;  
}
```

// Entity — 组件的容器

```
class Entity {  
    public string id;  
    public Position position;  
    public Velocity velocity;  
}
```

// System — 纯逻辑

```
class MovementSystem {  
    public void Update(Entity entity, float deltaTime) {  
        if (entity.position != null && entity.velocity != null) {  
            entity.position.x += entity.velocity.dx * deltaTime;  
            entity.position.y += entity.velocity.dy * deltaTime;  
        }  
    }  
}
```

8.4 ECS 的优势

[p.23]

1. **连续内存布局 (Cache-Friendly)**: 相同类型的组件在内存中连续存储, CPU 缓存可以高效预取。这是 ECS 最重要的性能优势。
2. **更少的不必要数据**: 实体只拥有需要的组件, 不携带无关数据。
3. **并行性**: System 只操作组件数据, 没有对象间复杂的引用关系, 更容易安全地多线程并行。

8.5 Unity 中的 ECS: DOTS

[p.24]

Unity 近年推出了自己的 ECS 系统 **DOTS (Data-Oriented Technology Stack)**, 包含:

- **Entities 包**——ECS 核心
- **Jobs System (C# Job System)** ——安全的多线程
- **Burst Compiler** ——将 C# 代码编译为高度优化的原生代码

DOTS 能让性能提升数倍到数十倍, 特别适合有大量实体的游戏 (RTS、大世界、粒子系统等)。

9 课后习题详解

[p.25]

9.1 题 1: 手动编写观察者模式系统

```
public interface IObserver {  
    void OnNotify(string eventName, object data);  
}  
  
public class Subject {  
    private List<IObserver> observers = new List<IObserver>();  
  
    public void Register(IObserver observer) {  
        if (!observers.Contains(observer))  
            observers.Add(observer);  
    }  
  
    public void Unregister(IObserver observer) {  
        observers.Remove(observer);  
    }  
}
```

```

    }

    public void Notify(string eventName, object data = null) {
        // 逆向遍历防止遍历中删除导致的问题
        for (int i = observers.Count - 1; i >= 0; i--) {
            observers[i].OnNotify(eventName, data);
        }
    }
}

```

// 使用示例

```

public class Player : MonoBehaviour, IObservable {
    void Start() {
        GameManager.instance.eventSubject.Register(this);
    }

    public void OnNotify(string eventName, object data) {
        if (eventName == "EnemyKilled") {
            int score = (int)data;
            Debug.Log($"获得分数: {score}");
        }
    }
}

```

9.2 题 2: 编写 *MonoSingleton* 基类

```

public class MonoSingleton<T> : MonoBehaviour where T : MonoBehaviour {
    private static T _instance;
    public static T Instance {
        get {
            if (_instance == null) {
                _instance = FindObjectOfType<T>();
                if (_instance == null) {
                    GameObject go = new GameObject(typeof(T).Name);
                    _instance = go.AddComponent<T>();
                }
            }
            return _instance;
        }
    }
}

```

```
protected virtual void Awake() {
    if (_instance == null) {
        _instance = this as T;
        DontDestroyOnLoad(gameObject); // 跨场景保持
    } else if (_instance != this) {
        Destroy(gameObject);           // 重复实例则销毁
    }
}
}
```

与普通 Singleton 的关键区别:

- 继承自 MonoBehaviour, 可以挂到 GameObject 上使用 Unity 生命周期
- 使用 FindObjectOfType<T>() 查找场景中已有实例
- DontDestroyOnLoad 确保场景切换时单例不丢失
- Awake() 中处理重复实例 (场景中有两个时自动销毁多余)

9.3 题 3: 指令模式实现移动与撤回

```
public interface ICommand {
    void Execute();
    void Undo();
}

public class MoveCommand : ICommand {
    private Transform player;
    private Vector3 direction;

    public MoveCommand(Transform p, Vector3 dir) {
        player = p;
        direction = dir;
    }

    public void Execute() { player.position += direction; }
    public void Undo()    { player.position -= direction; }
}

public class PlayerController : MonoBehaviour {
    private Stack<ICommand> history = new Stack<ICommand>();
    public float moveDistance = 1f;
```

```

void Update() {
    if (Input.GetKeyDown(KeyCode.W)) {
        var cmd = new MoveCommand(transform, Vector3.forward * moveDistance);
        Execute(cmd);
    }
    if (Input.GetKeyDown(KeyCode.S)) {
        var cmd = new MoveCommand(transform, Vector3.back * moveDistance);
        Execute(cmd);
    }
    if (Input.GetKeyDown(KeyCode.Z)) {
        if (history.Count > 0) {
            history.Pop().Undo();
        }
    }
}

void Execute(ICommand cmd) {
    cmd.Execute();
    history.Push(cmd);
}
}

```

9.4 题 4: 备忘录模式 (Memento Pattern)

备忘录模式用于捕获对象当前状态，以便之后恢复。

```

public class PlayerMemento : MonoBehaviour {
    private Vector3 initialPosition;
    private Quaternion initialRotation;
    private Vector3 initialScale;

    void Start() {
        // 记录初始状态
        initialPosition = transform.position;
        initialRotation = transform.rotation;
        initialScale = transform.localScale;
    }

    void Update() {
        if (Input.GetKeyDown(KeyCode.R)) {

```

```

        Restore();
    }
}

public void Restore() {
    transform.position = initialPosition;
    transform.rotation = initialRotation;
    transform.localScale = initialScale;
}

// 如需支持保存任意时刻的状态:
public PlayerState SaveState() {
    return new PlayerState {
        pos = transform.position,
        rot = transform.rotation,
        scale = transform.localScale
    };
}

public void LoadState(PlayerState state) {
    transform.position = state.pos;
    transform.rotation = state.rot;
    transform.localScale = state.scale;
}
}

[System.Serializable]
public class PlayerState {
    public Vector3 pos;
    public Quaternion rot;
    public Vector3 scale;
}

```

备忘录模式 vs 指令模式:

- 指令模式: 记录”做了什么” (操作序列), 通过反向操作撤销
- 备忘录模式: 记录”当时的快照” (状态), 通过恢复快照还原
- 备忘录更简单但可能占用更多内存 (每个快照都保存完整状态), 适合”回到初始值”这类场景